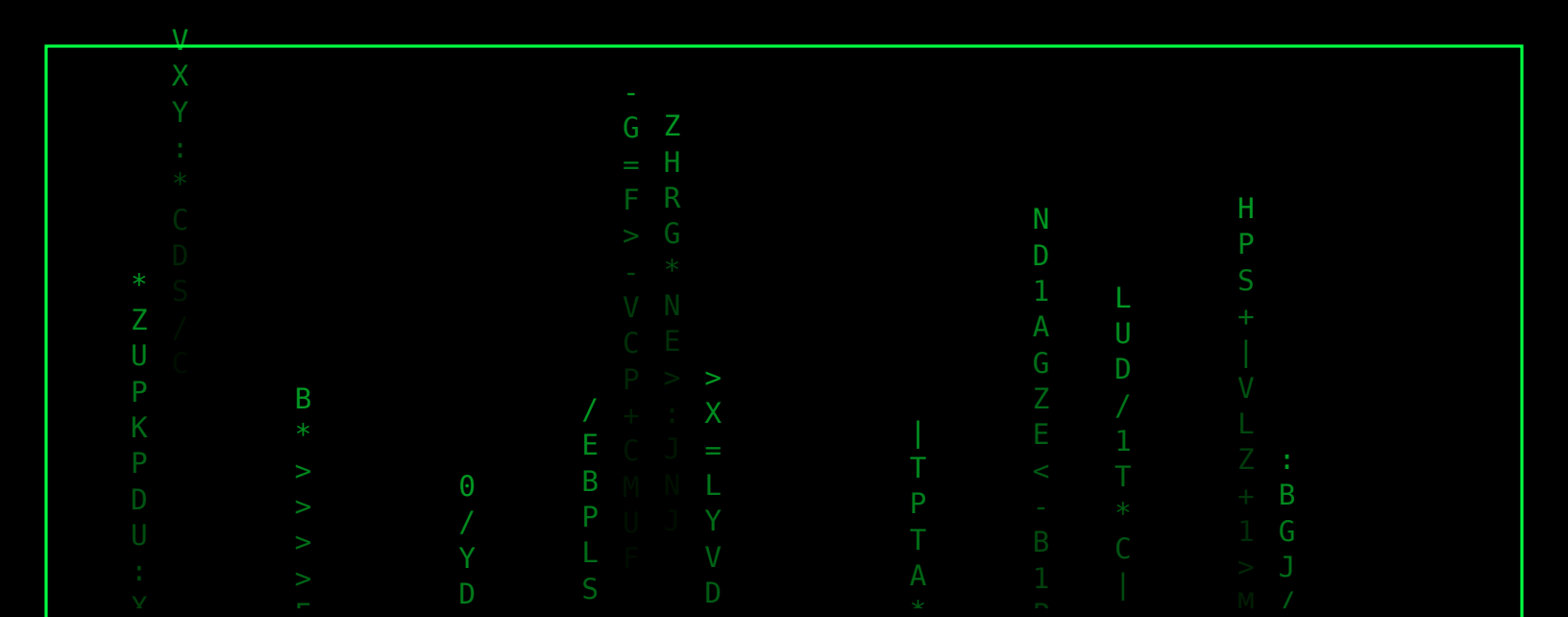
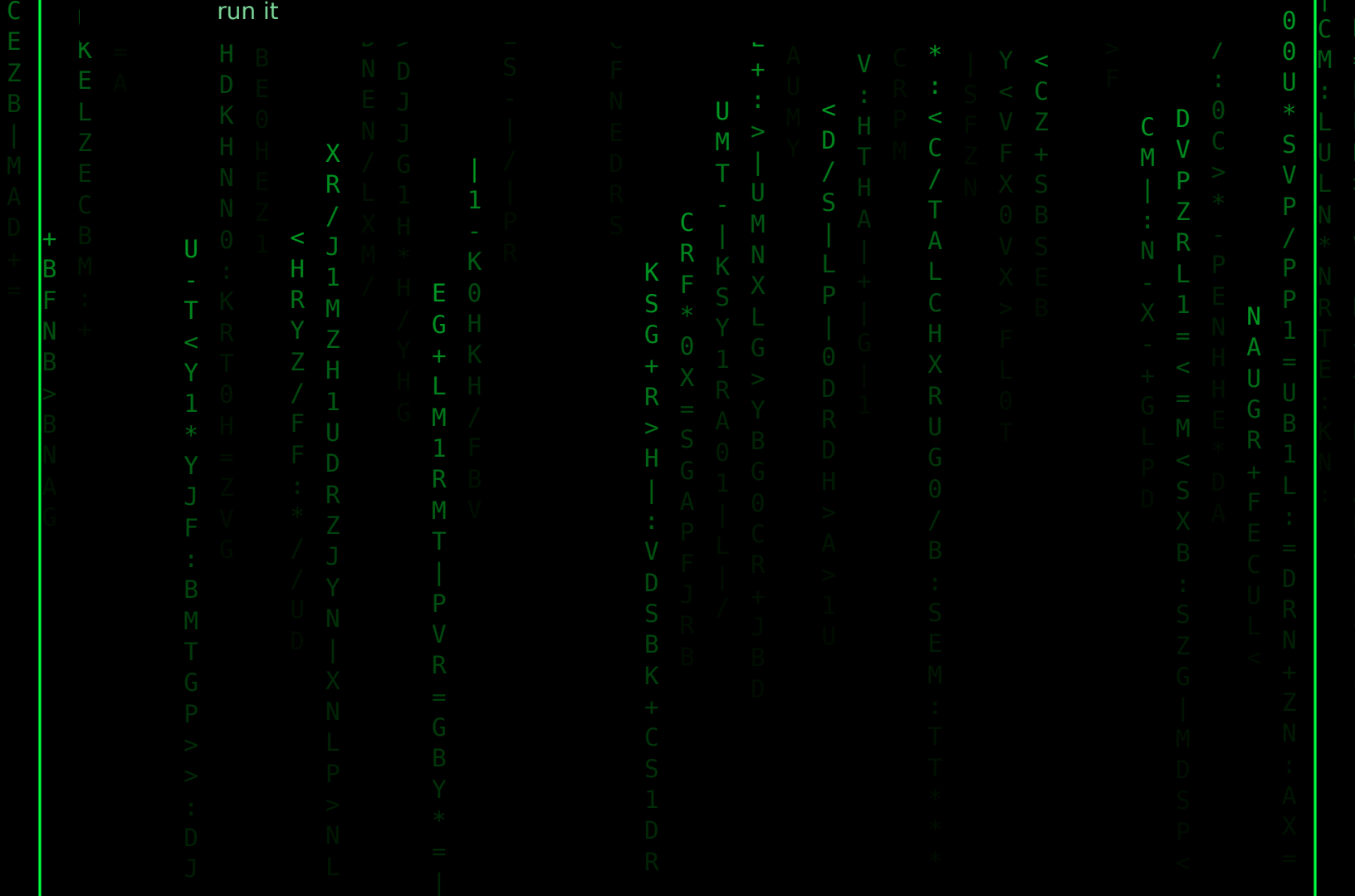


heg shell

A Unix shell, a Matrix terminal window and a kernel-portable core, written from scratch in C

Complete project guide: what it is, how it works, the concepts behind it, and how to run it



Contents

1. What this project is 3

2. What you can use it for 3

3. Project layout 4

4. Features 4

5. The concepts underneath 5

5.1 Processes: fork, exec, wait . . . 5

5.2 File descriptors . . . 5

5.3 Pipes . . . 6

5.4 Redirection and why order matters . . . 6

5.5 Signals . . . 6

5.6 Process groups and job control . . . 6

5.7 Terminals, raw mode and pseudo-terminals . . . 7

5.8 Parsing and grammar . . . 7

5.9 Expansion at execution time . . . 7

5.10 Testing methodology . . . 7

6. How heg works inside 8

6.1 The pipeline of stages . . . 8

6.2 Details worth knowing . . . 8

7. hegterm: heg's own window 9

7.1 Escape sequences understood . . . 9

7.2 The Matrix look . . . 9

7.3 Controls . . . 10

8. heg-lite and the toy kernel 10

8.1 What heg-lite supports . . . 10

8.2 The 12-operation contract . . . 10

8.3 Bring-up order that works . . . 11

8.4 Common pitfalls . . . 11

9. Tests 11

10. How to run it 12

Troubleshooting . . . 13

11. A guided tour to try 13

12. Limitations and next steps 14

Known limitations . . . 14

Next steps, in order of value . . . 14

13. Questions you should be able to answer 14

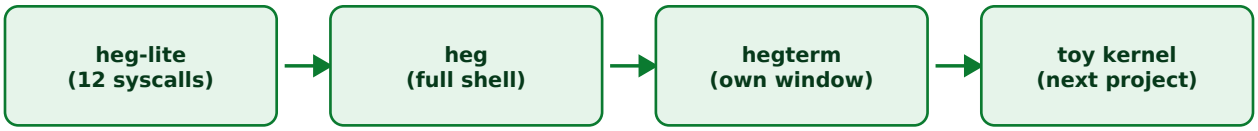
14. Glossary 15

1. What this project is

heg shell is a Unix command-line shell written from scratch in C. A shell is the program that reads what you type, such as `ls | grep foo > out.txt`, and asks the operating system to carry it out. Bash and zsh are shells. Almost every programmer uses one every day without seeing how it works; heg is a complete, working version where every line is yours to read.

The project has three layers that build on each other:

heg	The full shell (`heg.c`, about 3,700 lines of C, libc only). Pipelines, redirection, scripting, functions, job control, its own line editor with Tab completion. Runs on Linux and macOS.
hegterm	A terminal window of its own (`term/`, about 1,100 lines, uses SDL2). Black screen, neon-green text, falling digital rain. It runs heg, or any other program, inside it.
heg-lite	A 244-line freestanding core of the shell (`kernel/`) that needs only 12 operating-system operations and no standard library. It is the part meant to run on a toy kernel.



How the layers fit together: the same fork/exec/pipe design runs from a bare kernel up to a windowed terminal.

2. What you can use it for

- **As a real shell.** Run `./heg` and work in it: history with the arrow keys, Tab completion, colored prompt, Ctrl-C and Ctrl-Z job control.
- **As a scripting language.** `./heg script.heg` runs files with functions, loops, `$(...)` command substitution, arithmetic and here-documents.
- **As its own terminal.** `./hegterm` gives you a Matrix-style window. It is a general terminal, so `./hegterm -- bash` runs any program inside it.
- **As a learning tool.** The code is small enough to read end to end. Running `strace -f ./heg -c 'echo hi | wc'` shows the real `fork`, `pipe`, `dup2` and `execve` calls the shell makes.
- **As the base for other projects.** It feeds directly into a fuzzing project (fuzz the tokenizer and parser) and into a toy-kernel project (heg-lite is the first user program the kernel should run).

> heg is not trying to replace bash. Bash has decades of features. The value of heg is that you understand every line of it, and that it shows how the pieces of a Unix system fit together.

3. Project layout

File	Lines	What it is
heg.c	3,666	The full shell: tokenizer, expansion, parser, job control, builtins, executor, line editor, REPL
Makefile		Targets: all, test, test-asan, hegterm, test-term, hegl, test-lite, dist, clean
test_shell.sh	177	90 differential checks: each snippet runs in bash and in heg and must match
test_jobs.py	166	Job-control tests driven through a real pseudo-terminal (13 steps)
test_editor.py	210	Line-editor tests through a pseudo-terminal: history, cursor keys, Tab (18 steps)
term/term.c, term.h	525	Screen model: grid of cells plus the ANSI/VT escape-sequence parser. No SDL, no OS calls
term/hegterm.c	587	SDL2 front-end: window, fonts, keyboard, pty, Matrix theme, digital rain
term/test_term.c	199	33 tests for the screen model, plus heg running inside a pty
kernel/hegl.c	244	heg-lite: the freestanding, kernel-portable shell core
kernel/hegl_sys.h	115	The 12-operation OS interface, with POSIX and xv6 back-ends
kernel/test_hegl.sh	19	heg-lite checked against bash (12 checks)
kernel/KERNEL_PORT.md		Step-by-step guide to running heg-lite on your own kernel
README.md		Short project overview

4. Features

Area	What is supported
Pipelines	`a | b | c`, up to 32 stages
Redirection	`< > >> N> N>> N< N>&M &>`, applied strictly left to right. Here-documents `<<EOF` and `<<'EOF'`
Lists	`; && ||` with bash-compatible short-circuit behavior
Control flow	`if / elif / else / fi`, `while`, `until`, `for ... in`, `break [n]`, `continue [n]`
Functions	`name() { ...; }`, `return`, `local`, positional parameters `$1 $@ $* $#` and `shift`
Grouping	Subshells `( ... )` and groups `{ ...; }`
Expansion	Quotes, `$VAR`, `${VAR}`, `${VAR:-x}`, `${VAR:=x}`, `${VAR:+x}`, `${#VAR}`, `$(cmd)`, backticks, `$((arith))`, `$? $$ $!`, `~`, globbing, field splitting
Assignments	`VAR=value` and one-shot `VAR=x cmd`

Area	What is supported
Background and jobs	`&`, `$!`, `wait`, process groups, Ctrl-C, Ctrl-Z, `jobs`, `fg`, `bg`, `kill %n`
Line editor	Arrow keys, Home/End, Delete, Ctrl-A/E/K/U/W/L, persistent history (`~/.heg_history`), Tab completion of commands, files and directories
Builtins	`cd pwd exit echo export unset jobs fg bg wait kill history source . help true false : test [ read eval type shift local return break continue`
Scripting	`heg script.heg`, `heg -c 'cmd'`, commands from stdin, `#` comments, multi-line input with a `>` continuation prompt

5. The concepts underneath

This section explains the operating-system and language ideas that heg uses. Each one is a standard topic in an operating-systems or compilers course, and each appears directly in the code.

5.1 Processes: fork, exec, wait

A process is a running program with its own memory and open files. Unix creates processes in two steps, and a shell is the best way to see them:

1. `fork()` copies the calling process. Afterwards there are two nearly identical processes: the parent and the child. `fork` returns 0 in the child and the child's id in the parent.
2. `exec()` replaces the child's program with a new one, for example `ls`. The process id stays the same; the memory and code are replaced.
3. `wait()` makes the parent sleep until the child finishes and returns its exit status. The shell stores the status in `$?`.

```
pid = fork();
if (pid == 0) {            /* child */
    execvp("ls", argv);    /* becomes ls; only returns on failure */
    _exit(127);            /* command not found */
}
waitpid(pid, &status, 0);  /* parent waits, then sets $? */
```

Exit codes follow a convention heg tests for: 0 means success, 127 means the command was not found, 126 means it was found but is not executable, and 128 + n means the process was killed by signal n (so Ctrl-C gives 130).

5.2 File descriptors

A file descriptor is a small integer that refers to an open file, pipe or terminal. Every process starts with three: 0 is standard input, 1 is standard output, 2 is standard error. Programs just read from 0 and write to 1, and do not know or care whether those are the keyboard, a file, or a pipe. The shell changes what the numbers point to before running the program, and that is the whole trick behind redirection and pipes.

5.3 Pipes

`pipe()` creates a kernel buffer with two ends: a read end and a write end. For `a | b | c` the shell creates two pipes, forks three children, and in each child connects the right pipe ends to stdin and stdout with `dup2`:

```
child i:   dup2(pipe[i-1].read  -> 0)    if it is not the first command
           dup2(pipe[i].write   -> 1)    if it is not the last command
           close EVERY pipe end          <- forgetting this makes pipelines hang
           apply redirections, then exec
```

> A reader only sees end-of-file when every copy of the write end is closed. If the shell or any child keeps a stray copy open, `a | b` waits forever. This is the classic pipeline bug, and the test suite includes `yes | head -3` specifically to catch it.

5.4 Redirection and why order matters

`cmd > file` opens the file and makes descriptor 1 point to it before `exec`. heg stores redirections as an ordered list and applies them left to right, exactly like bash:

```
cmd 2>&1 >/dev/null     # stderr -> the old stdout (terminal/pipe), stdout -> /dev/null
cmd >/dev/null 2>&1     # both stdout and stderr -> /dev/null
```

Builtins such as `cd` and `echo` run inside the shell process itself, so heg saves descriptors 0 to 2, applies the redirections, runs the builtin, then restores them. A here-document is implemented by writing its text into a pipe and connecting the read end to stdin (limit 16 KiB).

5.5 Signals

A signal is a small message the kernel delivers to a process: `SIGINT` (Ctrl-C), `SIGTSTP` (Ctrl-Z), `SIGCHLD` (a child changed state), `SIGPIPE` (writing to a pipe nobody reads). The shell must ignore `SIGINT`, `SIGTSTP`, `SIGTTIN` and `SIGTTOU` itself so that Ctrl-C kills the running job but never the shell. Children reset those signals to the default before `exec`, so the programs you run behave normally.

5.6 Process groups and job control

Typing Ctrl-C sends a signal to the whole foreground process group, not to one process. That is why a pipeline can be stopped as a unit. heg puts every pipeline in its own group with `setpgid`, and hands the terminal to the foreground group with `tcsetpgrp`. Details that make it work:

- `waitpid(..., WUNTRACED)` reports both exits and stops, which is how Ctrl-Z turns into a "Stopped" job.
- `setpgid` is called in both the parent and the child to avoid a startup race.
- Finished background jobs are reaped with `WNOHANG` before each prompt, so there are no zombie processes, and heg announces `[1]+ Done ...`.
- Subshells and `$(...)` run with job control switched off, so Ctrl-C reaches the processes inside them.

5.7 Terminals, raw mode and pseudo-terminals

A terminal normally works in line mode: the kernel collects what you type, lets you backspace, and hands the line to the program on Enter. To implement history, arrow keys and Tab completion, heg's line editor switches the terminal to raw mode with `termios`, reads every keypress itself, decodes escape sequences such as `ESC [ A` (up arrow), and redraws the line.

A pseudo-terminal (pty) is a fake terminal made of two connected ends. A program such as hegterm holds the master end and the shell holds the slave end, so the shell believes it is talking to a real terminal. This is how every terminal emulator, ssh server and tmux works, and it is how the job-control and editor tests drive heg.

5.8 Parsing and grammar

Turning text into something executable is a small compiler front end. heg uses a hand-written recursive-descent parser for this grammar:

```
list      := pipeline ( (';' | '&&' | '||' | '&' | newline) pipeline )*
pipeline  := command ( '|' command )*
command   := simple | compound | function
compound  := if ... fi | while/until ... done | for ... done | '(' list ')' | '{' list ';' '}'
simple    := ( assignment | word | redirection )+
```

The tokenizer can also report that input is **incomplete** (an open quote, a trailing `|`, an `if` without `fi`, a pending here-document). The interactive loop then shows a `>` prompt and asks for more lines. The parser returns one of three results: success, syntax error, or incomplete.

5.9 Expansion at execution time

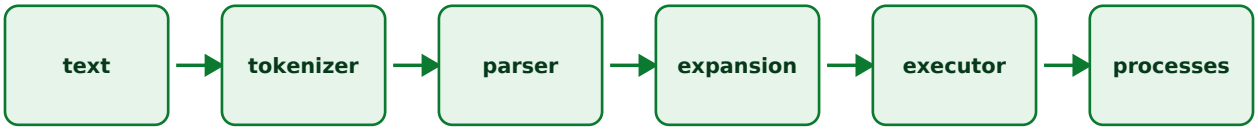
Words such as `$HOME`, `$(date)`, `$((2+3))` and `*.c` are expanded when a command runs, not when it is parsed. That matters: a loop body must expand again on every iteration, and `cd /tmp && echo $PWD` must see the new directory. heg expands into a fresh, separate structure so the parsed tree is never modified, which lets loops and functions run many times. Unquoted results are split into fields and globbed; quoted results are not.

5.10 Testing methodology

- **Differential testing:** run the same snippet in bash and in heg and require identical output and exit status. Bash acts as the specification.
- **Pseudo-terminal tests:** job control and the line editor cannot be tested without a tty, so Python drives heg through a pty and presses Ctrl-Z, Ctrl-C, arrow keys and Tab like a user.
- **Sanitizers:** AddressSanitizer and UBSan builds catch memory errors and undefined behavior. The suites run clean.
- **Fuzzing the screen model:** 20,000 random byte strings are fed to the terminal's escape-sequence parser, and the cursor must never leave the screen.

6. How heg works inside

6.1 The pipeline of stages



Every input line goes through these stages. Expansion and execution happen together, command by command.

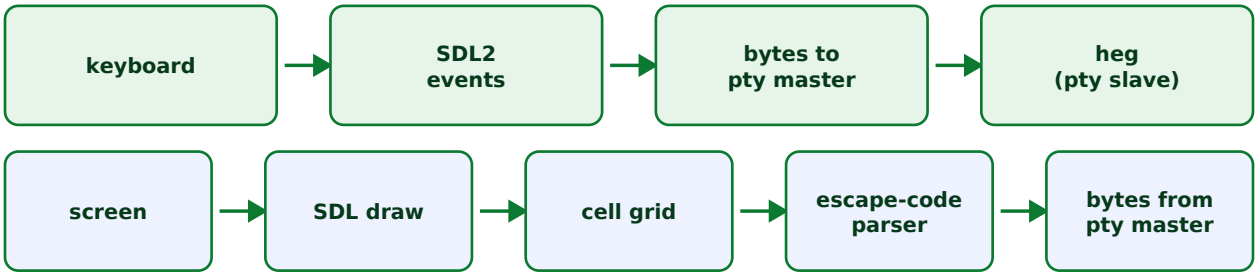
Stage	What it does	Key functions
Tokenizer	Splits input into words and operators, honors quotes and escapes, reports incomplete input	`tokenize`
Parser	Builds lists, pipelines and compound commands; returns ok / syntax error / incomplete	`parse`, `parse_list`, `parse_pipeline`, `parse_command`
Expansion	Parameters, command substitution, arithmetic, tilde, field splitting, globbing	`expand_word`, `expand_var`, `expand_braced`, `arith_eval`, `command_substitute`
Executor	Forks, wires pipes, applies redirections, runs builtins in-process, handles break/continue/return	`run_pipeline`, `run_child`, `run_inproc`, `run_list`, `run_compound`, `call_function`
Job control	Process groups, terminal hand-off, waiting, stop/continue, job table	`signal_job`, `find_job_by_pid`
Line editor	Raw-mode editing, history, completion, redraw	`editor_readline`, `complete_word`, `tty_readline`

6.2 Details worth knowing

- **Functions are stored by the function table, which takes ownership of the parsed body.** Calling a function runs the stored tree with new positional parameters; `local` variables are restored when the function returns.
- **Control flow uses flags.** `break`, `continue` and `return` set a control flag and a count (`ctl`, `ctl_n`); enclosing loops and functions check it after every command, which makes `break 2` work.
- **Assignments are detected before splitting.** `x=$(echo a b c)` keeps its value whole; without that check it would be split into separate words.
- **Input sources are unified.** `-c`, script files, stdin and the interactive terminal all feed the same driver, so a script behaves the same however it is run.
- **Dumb terminals fall back.** If `TERM` is `dumb` or input is not a tty, the editor is skipped and lines are read plainly, which keeps pipes and tests simple.
- **Prompt and history.** The prompt shows the current directory and is colored by the last exit status. History persists in `~/.heg_history`.

7. hegterm: heg's own window

hegterm is a terminal emulator: the program that draws the window you normally get from Ubuntu's Terminal app. It has two halves, deliberately kept separate.



Top: input path. Bottom: output path, read from right to left. The pty makes heg believe it runs in a real terminal.

Part	File	Responsibility
Screen model	`term/term.c`	A grid of cells (character, colors, bold/underline/reverse) and a byte-at-a-time state machine for ANSI/VT escape sequences. Scrollback ring of 5,000 lines. No SDL and no OS calls, so it is fully testable without a display.
Front-end	`term/hegterm.c`	`forkpty` to start heg, key-to-escape-sequence translation, glyph texture cache, window resize (`TIOCSWINSZ`), the Matrix theme and rain.

7.1 Escape sequences understood

- Cursor movement and positioning (`A B C D E F G H d f`), save/restore cursor.
- Erase (`J K X`), insert and delete characters and lines (`@ P L M`), scrolling (`S T`, reverse index) and scroll regions.
- Colors and attributes (SGR): 16 colors, 256-color palette, true color, bold, underline, reverse.
- Alternate screen (`?1049`), cursor visibility (`?25`), bracketed paste, UTF-8 text.
- Replies to the shell: cursor position report (`ESC[6n`) and device attributes (`ESC[c`).
- Unknown sequences are parsed and ignored, so they never corrupt the screen.

7.2 The Matrix look

- Black background, neon green (#00ff41) default text, a glowing halo drawn around each letter, faint CRT scanlines and a green frame.
- Digital rain falls behind the text. It is brightest for the first seconds, then settles to a calm background, and never draws left of the last character on a row so text stays readable.
- Every ANSI color is mapped to a shade of green with the same brightness order, so `ls --color` still shows differences. `--true-colors` switches the real colors back on.

7.3 Controls

Key or option	Effect
F2 / F3	Toggle the digital rain / toggle glow and scanlines
Ctrl + and Ctrl -	Zoom the font
Mouse wheel, Shift+PgUp, Shift+PgDn	Scroll back through history
Ctrl+Shift+V	Paste from the clipboard
`-s N`	Font size, for example `./hegterm -s 22`
`--no-rain`, `--true-colors`	Start without rain / with real ANSI colors
`-- command args`	Run another program inside the window, for example `./hegterm -- bash`

> Not implemented yet: selecting and copying text with the mouse, wide (CJK) characters, and mouse reporting for full-screen programs such as vim.

8. heg-lite and the toy kernel

The full shell needs a complete POSIX system: signals, termios, job control, malloc, libc. A hobby kernel has none of that at first. **heg-lite** is the same fork/exec/pipe design, reduced to what a young kernel can provide. All memory is static, and the only dependency is one header, `hegl_sys.h`.

8.1 What heg-lite supports

- Pipelines (`a | b | c`, up to 8 stages), redirection (`< > >>`), background (`&`), sequencing (`;`), single and double quotes, PATH lookup, `#` comments.
- Builtins: `cd`, `pwd`, `exit`, `help`. It prints a `heg>` prompt only when input is a terminal.

8.2 The 12-operation contract

Operation	Kernel call it needs
`hs_fork`, `hs_exec(path, argv)`, `hs_wait(&status)`, `hs_exit(code)`	`fork`, `exec`, `wait` (any child, -1 when none), `exit`
`hs_open(path, mode)`	`open` for read, create+truncate, append. Must return the lowest free descriptor
`hs_close`, `hs_read`, `hs_write`	`close`, `read`, `write`
`hs_pipe(fds)`, `hs_dup_to(src, dst)`	`pipe`; `dup2`, or `close(dst)` then `dup(src)` on kernels without dup2
`hs_chdir`, `hs_getcwd`	`chdir`; optional `getcwd`
`hs_isatty(fd)`	"Is this the console?" decides whether to print a prompt

The header has two implementations: a POSIX one so you can develop and test on Linux, and an xv6 one (`-DHEGL_XV6`) that maps each operation to xv6's system calls. Your kernel also needs a console driver that echoes typed characters and delivers input line by line, and a filesystem holding programs such as `/bin/echo`.

8.3 Bring-up order that works

1. `hs_write`: print the prompt.
2. `hs_read`: read a line from the keyboard.
3. `hs_fork`, `hs_exec`, `hs_wait`: run a simple command such as `echo`.
4. `hs_pipe` and `hs_dup_to`: pipelines.
5. `hs_open`: redirects.

8.4 Common pitfalls

- `open` must return the lowest free descriptor, or redirects land on the wrong number.
- A pipeline hangs if any process still holds a write end open. The kernel must release descriptors on `exit` and `exec`.
- `wait` must return -1 when there are no children, or the shell's wait loop spins forever.
- heg-lite expects exit statuses in the range 0 to 255; adapt `hs_wait` if your kernel packs status differently.

Why this pairing works: the shell is the first program that exercises almost every part of a kernel: process creation, address-space replacement, file descriptors, pipes, scheduling and blocking input. If heg-lite runs a three-stage pipeline on your kernel, your process and file layers work. Writing hegterm also teaches exactly what a kernel's own console driver has to do: take key presses, draw a grid of characters, and interpret control codes.

> heg-lite has been tested against bash on Linux (12 checks). It has not yet been run on xv6 or on your own kernel; that is the natural next milestone.

9. Tests

Command	Checks
`make test`	`test_shell.sh`: 90 differential checks against bash (pipes, redirection, lists, quoting, expansion, substitution, arithmetic, control flow, functions, groups, heredocs, builtins, background jobs, scripts, stdin, error codes). `test_jobs.py`: 13 pty steps (background job, `jobs`, `fg`, Ctrl-Z, `bg`, `kill %1`, Ctrl-C status 130, stopped pipeline). `test_editor.py`: 18 pty steps (history arrows, cursor editing, Ctrl shortcuts, Tab completion, continuation prompt, Ctrl-D).
`make test-asan`	The same three suites under AddressSanitizer and UBSan.
`make test-term`	33 checks on the screen model (escape sequences, scrollback, colors, UTF-8, resize, 20,000 random-byte feeds) plus heg running inside a pty and rendered by the model.

Command	Checks
`make test-lite`	heg-lite against bash: 12 checks.

Expected final lines: `passed: 90 failed: 0`, `passed: 13 failed: 0`, `passed: 18 failed: 0`, `passed: 33 failed: 0`, and `heg-lite: all 12 checks passed`.

10. How to run it

These steps assume Ubuntu in VirtualBox with the project zip downloaded to `~/Downloads/heg-shell.zip`. On native Linux or WSL the steps are the same from step 1.

Step 1: install the tools (once)

```
sudo apt update
sudo apt install -y build-essential zip unzip python3 gdb strace dos2unix \
                    libsdl2-dev libsdl2-ttf-dev
```

Step 2: unpack the project

```
cd ~
rm -rf ~/heg-shell
mkdir ~/heg-shell
unzip ~/Downloads/heg-shell.zip -d ~/heg-shell
cd ~/heg-shell
ls          # heg.c  Makefile  README.md  kernel  term  test_*.  ...
```

If `ls` shows only one folder, `cd` into it. Keep the project in your home folder (not on a shared or OneDrive folder) so file permissions survive.

Step 3: fix line endings and permissions

```
find . -type f \( -name "*.c" -o -name "*.h" -o -name "*.sh" -o -name "*.py" -o -name "Makefile" \) -exec dos2u
chmod +x test_shell.sh kernel/test_hegl.sh
```

Step 4: build

```
make            # builds ./heg
make hegl       # builds the kernel-portable ./kernel/hegl
make hegterm    # builds the Matrix window ./hegterm
```

Step 5: run

```
./heg           # the shell in the current terminal (Ctrl-D to exit)
./hegterm       # the Matrix window
./kernel/hegl   # heg-lite, prompt is "heg> "
```

Step 6: run the tests

```
make test
make test-term
make test-lite
make test-asan      # optional, slower
```

Step 7: package it again

```
make dist      # creates heg.zip
```

Troubleshooting

Problem	Fix
`make: command not found`	Step 1 did not finish; re-run the `apt install` line
`bad interpreter` or `^M` errors	Run the `dos2unix` command in step 3 again
`SDL2/SDL.h: No such file`	Install `libsdl2-dev libsdl2-ttf-dev`
`cannot open display`, no window	Run `./hegterm` from the desktop terminal, not over SSH
`no monospace font found`	`sudo apt install -y fonts-dejavu-core`, or set `HEGTERM_FONT=/path/to/font.ttf`
Window is slow in VirtualBox	Run `./hegterm --no-rain`; in VirtualBox set Display, Graphics Controller to VMSVGA and enable 3D acceleration
Unable to locate package	The VM has no internet; set the network adapter to NAT

11. A guided tour to try

Run these inside `./heg` or `./hegterm` and watch what happens.

```
echo hello | tr a-z A-Z                 # a pipeline
ls | wc -l
x=5; echo $((x * x))                    # arithmetic -> 25
for i in 1 2 3; do echo "n=$i"; done    # a loop
f() { echo "hi $1"; }; f world          # a function
fact() { if [ $1 -le 1 ]; then echo 1; else echo $(( $1 * $(fact $(($1-1))) )); fi; }
fact 10                                 # recursion through $(...) -> 3628800
cat <<EOF | sort
pear
apple
EOF
sleep 30        # press Ctrl-Z, then try:  jobs   bg   fg   (Ctrl-C to stop)
nosuchcmd; echo $?                      # 127
```

To watch the real system calls the shell makes, leave heg and run:

```
strace -f -e trace=clone,execve,pipe2,dup2,wait4 ./heg -c 'echo hi | wc'
```

You will see `pipe2` create the pipe, two `clone` calls (fork), `dup2` connect the ends, and `execve` start `echo` and `wc`. Reading this output next to `run_pipeline` in `heg.c` is the fastest way to understand the project.

A larger script that gave identical output in bash and heg: per-fruit totals from a file with functions and loops, FizzBuzz, recursive Fibonacci, globbing with heredocs, subshells with `||` recovery, and background jobs with `wait`.

12. Limitations and next steps

Known limitations

- No `case ... esac`, `select`, arrays, `[[ ]]`, aliases, `set -e` or process substitution `<(...)`.
- `"$@"` does not expand to separate quoted words.
- No Ctrl-R reverse history search; here-documents are limited to 16 KiB.
- `a && b &` backgrounds only `b` (bash backgrounds the whole list).
- Variables live in the process environment; there are no shell-only variables.
- hegterm: no mouse selection or copy, no wide characters, no mouse reporting.
- heg-lite: no scripting, no job control, no globbing; runs on POSIX and xv6 but not yet tested on a real kernel.

Next steps, in order of value

1. **Fuzz it.** Build with AFL++ and fuzz `tokenize` and `parse`. A shell parser is a good target, and "found and fixed N crashes" is a strong result.
2. **Fill the gaps.** `case`, `"$@"`, aliases, `set -e`, Ctrl-R. Re-run the bash comparison after each.
3. **Boot heg-lite on xv6**, then on your own kernel, following `kernel/KERNEL_PORT.md`.
4. **Add mouse selection and copy** to hegterm.
5. **Write a short design document** with the diagrams from this guide and a "what I learned" section.

13. Questions you should be able to answer

If you use this project in an application or interview, be ready to explain these without notes. The answers are all in this guide.

Question	Short answer
What happens when you type `ls | wc`?	Tokenize, parse into a pipeline of two commands, create one pipe, fork twice, in each child dup the pipe end onto 0 or 1, close all pipe fds, `exec`; the parent closes its ends and waits for both.
Why close every pipe end?	A reader sees EOF only when all write ends are closed; a leaked copy makes the pipeline hang.
Why does `2>&1 >file` differ from `>file 2>&1`?	Redirections apply left to right: the first copies stderr to the old stdout, the second copies it to the file.
Why does Ctrl-C not kill the shell?	The shell ignores SIGINT; children reset it to default before `exec`, and the foreground process group owns the terminal.
How does Ctrl-Z become a stopped job?	`waitpid` with WUNTRACED reports the stop; the shell records the job as stopped and takes the terminal back.

Question	Short answer
Why expand variables at execution time?	Loop and function bodies must re-expand each run, and earlier commands such as `cd` change what later words mean.
What is a pty and why use one?	A pair of connected ends that fakes a terminal, so a shell can run inside a program such as hegterm or a test script.
How do you know the shell is correct?	Differential tests against bash, pty tests for behavior that needs a terminal, and sanitizer runs.
Why is heg-lite separate from heg?	A young kernel lacks signals, termios and malloc; heg-lite keeps the fork/exec/pipe core behind 12 operations so it can run there.
What would you add next?	Fuzz the parser, then run heg-lite on a kernel you built.

14. Glossary

Term	Meaning
AST	Abstract syntax tree: the structure the parser builds from the text
Builtin	A command run inside the shell process itself, such as `cd`
dup2	Makes one file descriptor refer to the same thing as another
Escape sequence	A byte sequence starting with ESC that controls the terminal (move cursor, set color)
exec	Replaces the current process's program with another
File descriptor	A small integer naming an open file, pipe or terminal
fork	Creates a copy of the current process
Foreground group	The process group that currently owns the terminal and receives Ctrl-C
Fuzzing	Feeding a program large amounts of random or mutated input to find crashes
Job	A pipeline started by the shell, tracked so it can be stopped, resumed or killed
Pipe	A kernel buffer that connects one process's output to another's input
Process group	A set of processes that receive terminal signals together
pty	Pseudo-terminal: a software-made terminal with a master and a slave end
Raw mode	Terminal mode where the program receives every key press itself
Signal	A small message the kernel delivers to a process, such as SIGINT
Subshell	A child copy of the shell used for `( ... )` and `$(...)`
termios	The Unix interface for configuring terminal behavior
wait / waitpid	Waits for a child process to finish or stop
xv6	A small teaching Unix kernel from MIT, a good first target for heg-lite